

Projecteur — document de contexte pour Claude Code

Ce fichier est un document de **passation**. Il contient tout ce qui a été décidé, construit, et raté lors d'une première session de conception. Lis-le en entier avant de proposer du code.

1. Objectif

Application pour projeter des **versets bibliques** et des **cantiques** pendant les cultes, avec **intégration native à OBS Studio** pour la diffusion en ligne.

Contexte d'usage : ministère chrétien étudiant (CMR, Université Laval, Québec). L'auteur est le responsable technique du streaming des cultes. L'application doit donc servir **deux publics simultanément** : la salle (vidéoprojecteur) et les spectateurs en ligne (incrustation OBS).

Langue de travail et d'interface : **français**.

2. STRATÉGIE : web d'abord, desktop ensuite

Ceci est la décision structurante du projet. Lis cette section avant toute proposition technique.

La cible finale est une **application de bureau**. Mais la construction se fait en deux phases :

Phase	Forme	Objectif
1 — actuelle	Application web (React + Vite, aucun Tauri)	Valider le modèle de données, l'ergonomie de régie, le rendu projeté et l'overlay OBS
2 — ensuite	Enveloppe Tauri autour du même code	Vrai plein écran, offline garanti, SQLite, fiabilité

Pourquoi cet ordre

La phase 1 permet d'itérer en secondes plutôt qu'en minutes de compilation Rust, et de tester **immédiatement** ce qui compte : est-ce que la régie est agréable à utiliser en direct ? Est-ce que le texte est lisible au fond de la salle ? Est-ce que l'overlay OBS rend bien ?

Elle contourne aussi le bug de gestion de fenêtres Tauri qui a bloqué la première session (§8). Ce bug reste à résoudre, mais en phase 2, sans bloquer le reste.

Règle d'or de la phase 1

Écrire le code de sorte que la phase 2 ne soit qu'une enveloppe.

Toute la logique métier, le rendu et l'interface sont écrits en React/TypeScript standard, sans aucune dépendance à la plateforme. Les deux seuls points de contact avec

l'environnement — gestion des fenêtres et stockage — sont isolés derrière une **couche d'abstraction** (§5). Le jour où on ajoute Tauri, on écrit une seconde implémentation de cette couche et **rien d'autre ne bouge**.

Ce que la phase web ne pourra PAS valider

À savoir dès maintenant, ce sont précisément les raisons d'être de la phase 2 :

- Pas de vraie garantie de plein écran sans barre d'adresse (dépend de F11 et de la discipline de l'opérateur)
- Risque qu'un élément d'interface du navigateur apparaisse sur le vidéoprojecteur
- Pas de garantie offline si l'app est servie depuis un hébergement distant
- Pas d'accès disque, donc pas de SQLite ni de médias locaux
- Empreinte mémoire non représentative

Ne conclus donc pas de la réussite de la phase 1 que le desktop est inutile. Le desktop reste la cible.

3. Pourquoi ne pas simplement utiliser l'existant

Le marché est occupé (OpenLP, VideoPsalm, FreeShow, ProPresenter, EasyWorship). Le projet n'existe que parce que des manques précis ont été constatés à l'usage réel :

Outil testé	Frustration constatée
VideoPsalm	Aucune intégration OBS. Interface vieillissante.
FreeShow	Trop lourd, ralentit la machine quand OBS tourne en parallèle. Les noms de livres bibliques restent en anglais même quand le texte est en français.

Ces points **sont** le cahier des charges. Toute décision technique doit s'y rapporter.

Différenciation visée

1. **Intégration OBS de première classe** (Browser Source transparente + obs-websocket). Personne ne le fait bien côté francophone.
 2. **Localisation française complète**, y compris la nomenclature des livres.
 3. **Empreinte mémoire faible**, mesurée, avec OBS ouvert en parallèle.
-

4. Architecture — phase 1 (web)



Principe directeur — non négociable

La console décide, tout le reste affiche. La fenêtre de projection et l'overlay OBS ne calculent jamais leur propre état. Cette contrainte évite la classe de bugs la plus pénible en direct : deux écrans qui divergent.

Corollaire à appliquer : le composant de rendu (`Output`) est **partagé** entre l'aperçu de la console et la fenêtre de projection. L'aperçu est donc exact par construction, pas par ressemblance.

Synchronisation entre fenêtres

`BroadcastChannel` — API navigateur native, même origine, latence quasi nulle, aucun serveur requis. La console poste le `ShowState` complet à chaque changement.

Deux détails à ne pas oublier :

- **Rejeu à l'ouverture** : une fenêtre qui s'ouvre après coup n'a rien reçu. Elle poste un message `{ type: "hello" }` auquel la console répond en renvoyant l'état courant. Sans ça, ouvrir OBS en plein culte donne un écran vide.
- **Persistance** : la console sauvegarde le dernier état dans `localStorage`. Si elle est rechargée par accident, elle repart au bon endroit.

Les trois routes

Route	Rôle	Particularités
<code>/console</code>	Régie : déroulé, aperçu, raccourcis clavier	Seule à écrire dans le canal
<code>/projection</code>	Sortie vidéoprojecteur	Fond noir, curseur masqué, aucun élément d'interface
<code>/overlay</code>	Incrustation OBS	Fond transparent obligatoire

Overlay OBS en phase web

C'est le point où la phase 1 est **aussi bonne que la phase 2**, voire plus simple : OBS Browser Source consomme une URL. En développement, `http://localhost:5173/overlay`

fonctionne directement.

Impératif : `<html, body { background: transparent; }>`. Ne jamais poser de couleur de fond sur ces éléments, sinon l'incrustation masque la caméra.

Réglages OBS à documenter pour l'utilisateur : largeur 1920, hauteur 1080, décocher « Fermer la source quand invisible ».

Ouverture de la fenêtre de projection

Deux mécanismes, à implémenter dans cet ordre :

1. **Universel** — `window.open("/projection", "projection", "popup")`, puis l'opérateur glisse la fenêtre sur le vidéoprojecteur et appuie sur **F11**. Fonctionne partout, demande une manipulation.
2. **Automatique si disponible — Window Management API**
(`window.getScreenDetails()`), Chrome/Edge/Brave 100+, nécessite localhost ou HTTPS et une permission utilisateur). Permet de lister les écrans et d'ouvrir directement en plein écran sur le bon. Détecter la disponibilité et retomber sur le mécanisme 1 sinon.

5. La couche d'abstraction plateforme

C'est le pivot entre les deux phases. À écrire dès le premier jour.

Un module `src/platform/` exposant une interface unique, avec deux implémentations interchangeables :

ts

```
// src/platform/types.ts
export interface ScreenInfo {
  id: string;
  label: string;
  width: number;
  height: number;
  isPrimary: boolean;
}

export interface Platform {
  /** Liste les écrans. Peut renvoyer [] en web sans permission accordée. */
  listScreens(): Promise<ScreenInfo[]>;

  /** Ouvre ou déplace la sortie de projection sur l'écran donné. */
  openProjection(screenId?: string): Promise<void>;

  /** Ferme la sortie de projection. */
  closeProjection(): Promise<void>;

  /** URL à donner à OBS pour l'overlay. */
  overlayUrl(): string;

  /** Stockage persistant, clé/valeur JSON. */
  store: {
    get<T>(key: string): Promise<T | null>;
    set<T>(key: string, value: T): Promise<void>;
  };
}
```

- `src/platform/web.ts` — `window.open`, Window Management API, `localStorage`/IndexedDB
- `src/platform/tauri.ts` — **phase 2** — commandes Tauri, SQLite

Sélection à la construction via une variable d'environnement Vite
 (`import.meta.env.VITE_PLATFORM`), pas de détection à l'exécution.

Aucun autre fichier du projet ne doit importer d'API navigateur ou Tauri directement. Si tu te retrouves à écrire `window.open` ailleurs que dans `web.ts`, la phase 2 sera douloureuse.

6. Modèle de données

ts

```

type SlideKind = "verset" | "cantique";

interface Slide {
  kind: SlideKind;
  reference: string; // « Jean 3.16 · LSG » ou « Strophe 2 »
  body: string;      // texte projeté, \n conservés
}

interface Item {
  id: string;
  kind: SlideKind;
  label: string;      // titre dans le déroulé, JAMAIS projeté
  slides: Slide[];
}

interface ShowState {
  slide: Slide | null;
  visible: boolean;   // false = écran noir, contenu reste chargé
  itemIndex: number;
  slideIndex: number;
}

```

`visible: false` correspond à l'écran noir : le contenu reste chargé, il est seulement masqué. On le récupère d'une touche.

Raccourcis clavier

- `→` / `PageDown` / `Espace` : diapositive suivante (franchit les items)
- `←` / `PageUp` : précédente
- `B` : bascule écran noir

7. Parti pris visuel

Console sombre type cabine technique : encre bleutée (`#101219`), panneaux (`#181B24`), filets (`#262B38`), texte parchemin (`#EDE7DA`), bleu ardoise (`#7A93A0`), rouge tally (`#C8483C`).

Polices système uniquement — l'app doit démarrer hors ligne, aucune requête réseau au chargement. La personnalité vient de l'espacement et des détails, pas d'une police téléchargée.

Élément signature : un **rail tally** vertical rouge à gauche de l'aperçu, qui s'allume seulement quand quelque chose est réellement à l'antenne. Convention empruntée aux régies de télévision.

Projection : fond noir, texte blanc (`clamp(28px, 4.4vw, 76px)`), interligne 1.32, (`text-wrap: balance`), marge de sécurité écran de 6vh/8vw, curseur masqué.

Overlay OBS : bandeau bas avec **dégradé** et non un bloc opaque — le visage du prédicateur doit rester visible derrière. Liseré rouge en bas, transition 260 ms, (`text-shadow`) pour la lisibilité sur fond clair.

Filet de sécurité obligatoire : un (`<style>`) inline dans le (`<head>`) de la page de projection (`(html, body { background: #000; })`), appliqué avant tout JavaScript. Si le front plante, la salle voit du noir, jamais du blanc. Sur un vidéoprojecteur, c'est la différence entre « il ne se passe rien » et « ça a planté ». **Ne pas mettre ce filet sur** (`/overlay`), qui doit rester transparent.

8. Le bug Tauri non résolu — à traiter en PHASE 2

Conservé pour mémoire. **Ne pas s'y attaquer maintenant.**

Symptôme

Au clic sur « Projeter sur ... », une fenêtre s'ouvrait sur le second moniteur mais :

- entièrement **blanche**, le contenu React ne s'affichait jamais
- titre « **Projection (Ne répond pas)** »
- clic droit sans effet, DevTools inaccessibles, obligation de tuer l'application
- cadre de fenêtre Windows visible **malgré** (`decorations(false)`)

Élément de diagnostic décisif

(`http://localhost:1420/projection.html`) ouvert **dans Chrome** s'affichait correctement en **noir**. Le front-end n'était donc pas en cause.

Tauri n'ayant **qu'une seule boucle d'événements** pour toutes les fenêtres, « Ne répond pas » suggère un blocage de cette boucle, donc de toute l'application.

Ce qui a été tenté

Tentative 1 — (`open_projection`) était synchrone, donc exécutée sur le thread principal ; or créer une fenêtre a besoin de cette même boucle pour aboutir. Passage en (`async fn`).

Tentative 2 — Suppression de la création de fenêtre à l'exécution. Fenêtre (`projection`) déclarée dans (`tauri.conf.json`) avec (`"visible": false`), existant dès le démarrage ; la commande ne faisait plus que (`set_fullscreen(false)`) → (`set_position`) → (`show()`) → (`set_fullscreen(true)`).

⚠ Ces deux tentatives n'ont probablement JAMAIS été compilées

Découvert après coup : deux copies du projet coexistaient.

- (`C:\dev\projecteur`) — celle que (`npm run tauri dev`) compilait

- `%USERPROFILE%\OneDrive\Bureau\Projets\projecteur` — celle où les fichiers corrigés étaient déposés par erreur

Le verdict « ça n'a rien changé » n'est donc pas fiable. Refaire les deux tentatives proprement en phase 2, en vérifiant que le terminal affiche bien `Compiling projecteur v0.1.0` après chaque modification de `lib.rs`.

Questions de diagnostic restées sans réponse

À obtenir **avant** tout correctif :

1. Quand la projection est blanche, **la fenêtre console répond-elle encore** ? Si oui, blocage limité au webview. Si non, c'est la boucle d'événements. *Cette réponse tranche entre deux causes très différentes.*
2. Le terminal affiche-t-il quelque chose au moment du clic ?
3. Une erreur autre que `__TAURI_INTERNALS__` dans la console Chrome ?

Pistes non explorées

- Permissions `capabilities/default.json` insuffisantes pour la fenêtre `projection`
- Interaction entre le runtime tokio du serveur overlay et celui de Tauri
- Version de WebView2 sur la machine
- **Méthode recommandée** : réduire à un cas minimal — une fenêtre secondaire, un `<h1>` statique, aucun appel de commande, aucun plein écran. Si ça marche, rajouter les éléments un par un.

9. Détails de la phase 2 (pour anticiper, pas pour coder maintenant)

Stack visée : Tauri v2

Electron a été écarté : FreeShow est en Electron, chaque fenêtre embarque un Chromium complet, et c'est précisément la cause de la lourdeur reprochée. Avec OBS en parallèle, on additionne deux moteurs de rendu. Tauri réutilise le WebView2 déjà présent dans Windows : binaire ~10 Mo contre 150+, mémoire divisée par 3 ou 4.

Nuance honnête : si le bug §8 s'avère structurel et bloquant, **reconsidérer Electron est légitime** — son multi-écrans est plus mature et mieux documenté. Ne pas basculer avant un vrai diagnostic.

Serveur overlay local

En phase 2, l'overlay ne peut plus être servi par Vite. Il faut un serveur HTTP embarqué. Implémentation qui existait et fonctionnait :

- Crate `axum`, port de départ **4599**, incrémenté si occupé
- Propre `std::thread` avec runtime tokio dédié
- `GET /overlay` → page HTML transparente via `include_str!`

- `GET /events` → flux SSE du `ShowState` en JSON
- **Mémoire le dernier état** et le renvoie à chaque nouveau client, pour qu'OBS se reconnecte en plein culte sans intervention

Commandes Tauri prévues

Commande	Signature	Rôle
<code>list_monitors</code>	<code>() -> Vec<MonitorInfo></code>	index, name, width, height, is_primary
<code>open_projection</code>	<code>(monitor_index: usize)</code>	positionne et affiche la projection
<code>push_state</code>	<code>(payload: Value)</code>	emit + diffusion SSE
<code>current_state</code>	<code>() -> Option<Value></code>	rejoué par la projection à son ouverture
<code>overlay_port</code>	<code>() -> u16</code>	port réellement retenu

Dépendances Rust utilisées : `tauri 2`, `serde`, `serde_json`, `axum 0.7`, `tokio` (rt-multi-thread, net, macros, sync), `tokio-stream` (sync), `tower-http` (cors).

Icônes

`tauri-build` exige `src-tauri/icons/icon.ico` sur Windows, sinon échec avec `icons/icon.ico not found`. Générer via `npm run tauri icon app-icon.png` et renseigner `bundle.icon` dans `tauri.conf.json`.

10. Feuille de route

Phase 1 — web

1. **Squelette** : Vite + React + TS, trois routes, couche `platform/`
2. **Contenu en dur** dans `src/show.ts` — quelques versets et un cantique structuré en strophes. Aucune base de données pour l'instant.
3. **Console + projection** synchronisées par `BroadcastChannel`, raccourcis clavier, aperçu partagé
4. **Overlay OBS** transparent, testé pour de vrai dans OBS
5. **Données réelles** : Bible LSG en JSON, un recueil de cantiques
6. **Recherche et parseur de référence** (§11)
7. **Test en culte réel**, plan B prêt

Phase 2 — desktop

8. Enveloppe Tauri, seconde implémentation de `platform/`
9. Résolution du bug §8

10. SQLite via `tauri-plugin-sql`, migration des données

11. Serveur overlay embarqué

Ensuite

Setlist persistant, télécommande mobile, pilotage obs-websocket v5, thèmes (police, fond image/vidéo, zones sûres TV), écran retour musiciens, multilingue.

11. Conception des données bibliques

À trois tables séparées — c'est la correction du défaut constaté chez FreeShow, où les noms de livres sont codés en dur et seul le texte est traduit :

Table	Contenu
Texte biblique	LSG, Darby, Second 21... versions interchangeables
Nomenclature	Nom complet, nom court, abréviation — par langue
Alias de recherche	Toutes les façons de taper un livre

Exemple pour Jean : `jean`, `jn`, `john`, `jhn`, `43`. La recherche doit être **insensible aux accents et à la casse** : `genese`, `Genèse`, `1 corinthiens`, `1co`, `1 Co` doivent tous fonctionner.

Particularités francophones que les outils anglo-saxons ratent : « Cantique des cantiques », titres de Psaumes comptés comme verset 1 selon les éditions, numérotations Psaumes 9/10 divergentes.

En phase 1, ces trois tables existent en **JSON** et la recherche se fait en mémoire. La structure ne change pas en phase 2, seul le support devient SQLite.

Parseur de référence

Taper `jn 3:16` ou `Ps 23` doit afficher en **moins d'une seconde**. C'est le geste le plus fréquent de tout le logiciel, il doit être parfait.

Modèle cantique

titre, numéro, recueil, **strophes/refrain structurés** (surtout pas un bloc de texte), tonalité, auteur, référence CCLI. Prévoir l'**import depuis les formats OpenLP et VideoPsalm** — sans migration, personne ne changera d'outil.

12. Budget de performance

Cibles de la phase 2, à mesurer **OBS ouvert en parallèle**. En phase 1 la mesure mémoire n'est pas représentative, mais le temps de changement de diapositive l'est déjà.

Mesure	Cible
RAM au repos	< 200 Mo
RAM en culte (Bible + cantiques chargés)	< 400 Mo
Démarrage à froid	< 3 s
Changement de strophe	< 50 ms perçu

Si un ajout fait sauter le budget, il attend. Ce budget est la raison d'être du choix de Tauri ; le trahir vide le projet de son sens.

13. Contraintes légales

Versions bibliques. Segond 1910 et Darby sont dans le domaine public et peuvent être embarquées. **Segond 21, Bible du Semeur, NBS sont protégées** et exigent une licence. Formats de récupération : Zefania XML, OSIS, ou les dumps SQLite d'OpenLP.

Cantiques. Droits SACEM/CCLI sur les compositions récentes. À clarifier avant toute diffusion publique du logiciel.

14. Environnement et pièges à ne pas reproduire

Machine

- **Windows 11**, version 10.0.26200.8875
- **Node** avec npm fonctionnel
- **Rust 1.97.1**, toolchain `x86_64-pc-windows-msvc` (installé, inutile en phase 1)
- Build Tools C++ présents
- Navigateur : Brave/Chrome — compatible Window Management API

Emplacement de travail

`C:\dev\projecteur`, jamais ailleurs.

Les dossiers Bureau/Documents/Images de cette machine sont **redirigés vers OneDrive**. Le projet ne doit jamais y résider : `node_modules` (des dizaines de milliers de petits fichiers) et, en phase 2, `src-tauri/target` (2 à 5 Go) provoquent des verrous de fichiers et des erreurs de build incompréhensibles, en plus de saturer le quota.

Deux copies du projet

Cause de plusieurs heures perdues lors de la session précédente (§8). **Un projet, un seul emplacement.** Vérifier la barre d'adresse à chaque copie de fichier.

Git

N'a jamais été initialisé, et le projet a été supprimé une fois par accident. `git init` dès le premier lancement réussi. `.gitignore`: `node_modules`, `dist`, et en phase 2 `src-tauri/target`, `src-tauri/gen`.

15. Attentes de travail

L'auteur préfère des réponses **structurées, directes, actionnables**, sans questions préliminaires inutiles. Communication en **français**.

Une leçon de la session précédente : **ne propose pas de correctif avant d'avoir le diagnostic**. Deux correctifs ont été livrés à l'aveugle, aucun n'a fonctionné, et la vraie cause (deux copies du projet) n'a été trouvée que bien plus tard. Pose la question qui discrimine entre les hypothèses, obtiens la réponse, puis corrige.

Le risque principal du projet reste **le scope**. Le MVP doit être humiliant de simplicité : affichage d'un verset, cantique par strophe, un seul thème, écran noir. Rien d'autre avant d'avoir servi un vrai culte, plan B prêt.

Le concurrent réel n'est pas ProPresenter. C'est **la confiance du technicien**.